TEXAS A&M UNIVERSITY-SAN ANTONIO

CSCI 2436 Data Structures

Spring 2026

Chapter 7 Queues, Deques, and Priority Queues

Objectives

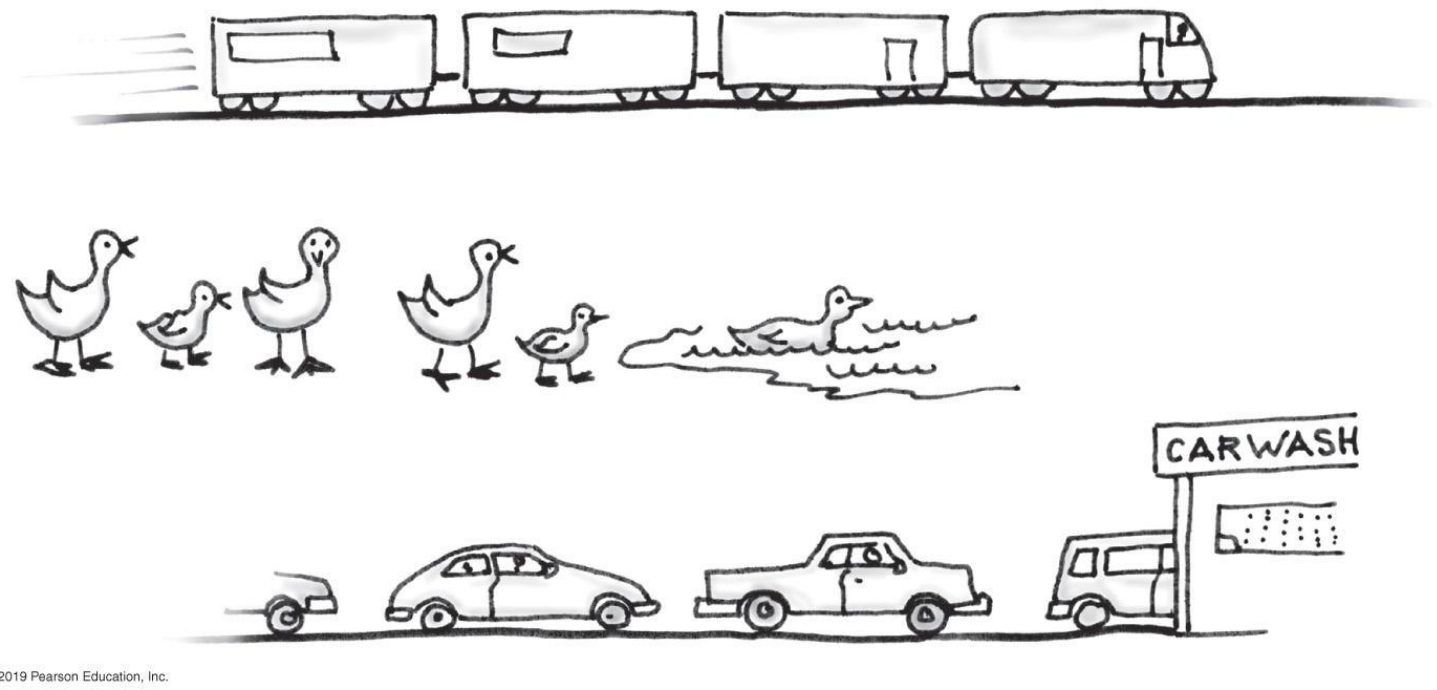
- Describe the operations of the queue
- Use a queue to simulate a waiting line
- Use a queue in a program that organizes data in a first-in, first-out manner
- Describe the operations of the priority queue

The ADT Queue

- A queue is another name for a waiting line
- Used within operating systems and to simulate real-world events
  - Come into play whenever processes or events must wait
- Entries organized **first-in, first-out (FIFO)**

The ADT Queue

- Some everyday queues



The ADT Queue

- Terminology
  - Item added first, or earliest, is at the front of the queue
  - Item added most recently is at the back of the queue
- Additions to a software queue must occur at its back
- Client can look at or remove only the entry at the front of the queue

The ADT Queue

- Data: collection of objects in chronological order and having the same data type

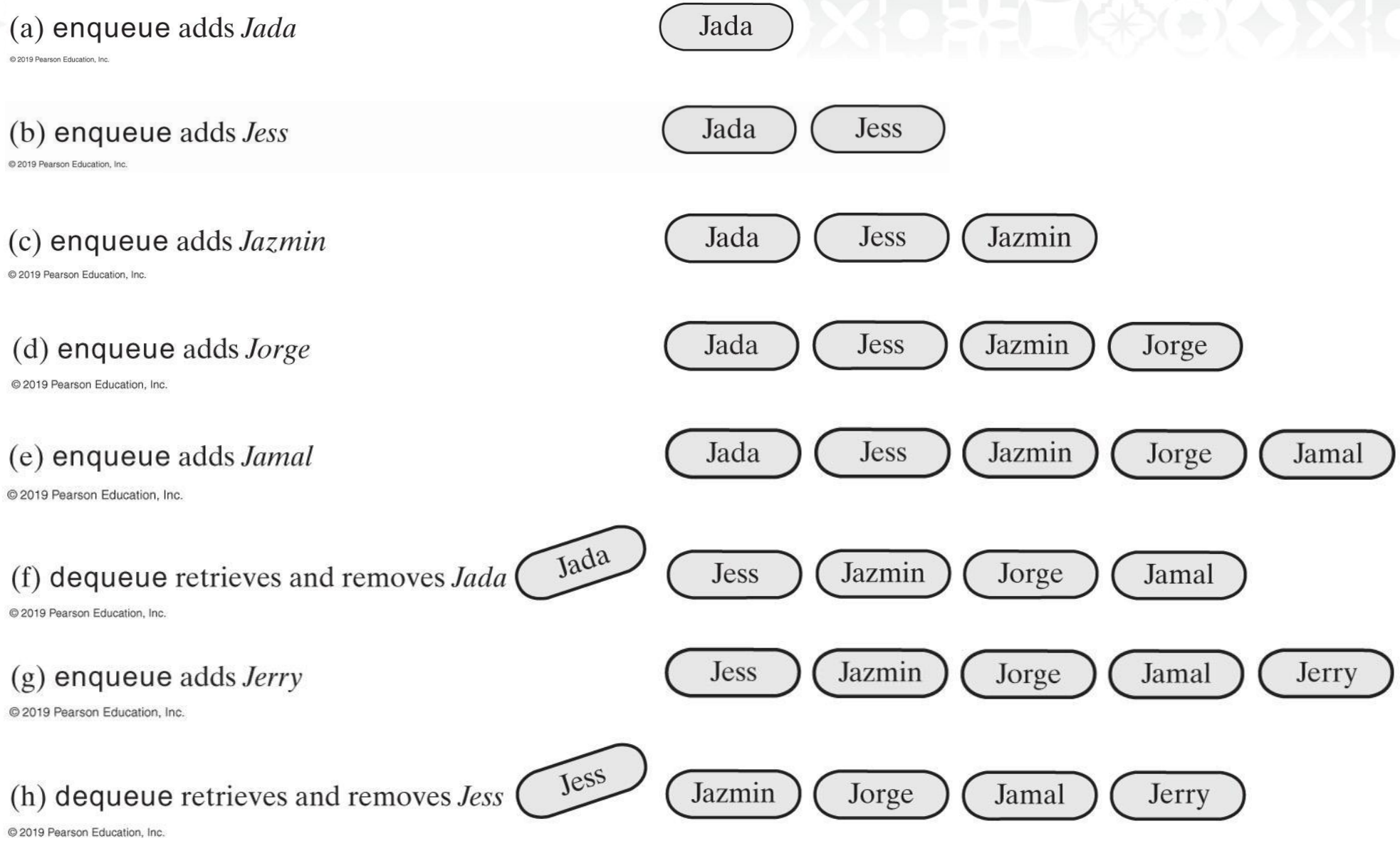
Pseudocode	UML	Description
enqueue(newEntry)	+enqueue(newEntry: integer): void	Task: Adds a new entry to the back of the queue. Input: newEntry is the new entry. Output: None.
dequeue()	+dequeue(): T	Task: Removes and returns the entry at the front of the queue. Input: None. Output: Returns the queue's front entry. Throws an exception if the queue is empty before the operation.
getFront()	+getFront(): T	Task: Retrieves the queue's front entry without changing the queue in any way. Input: None. Output: Returns the queue's front entry. Throws an exception if the queue is empty.
isEmpty()	+isEmpty(): boolean	Task: Detects whether the queue is empty. Input: None. Output: Returns true if the queue is empty.
clear()	+clear(): void	Task: Removes all entries from the queue. Input: None. Output: None.

The ADT Queue

- An interface for the ADT queue

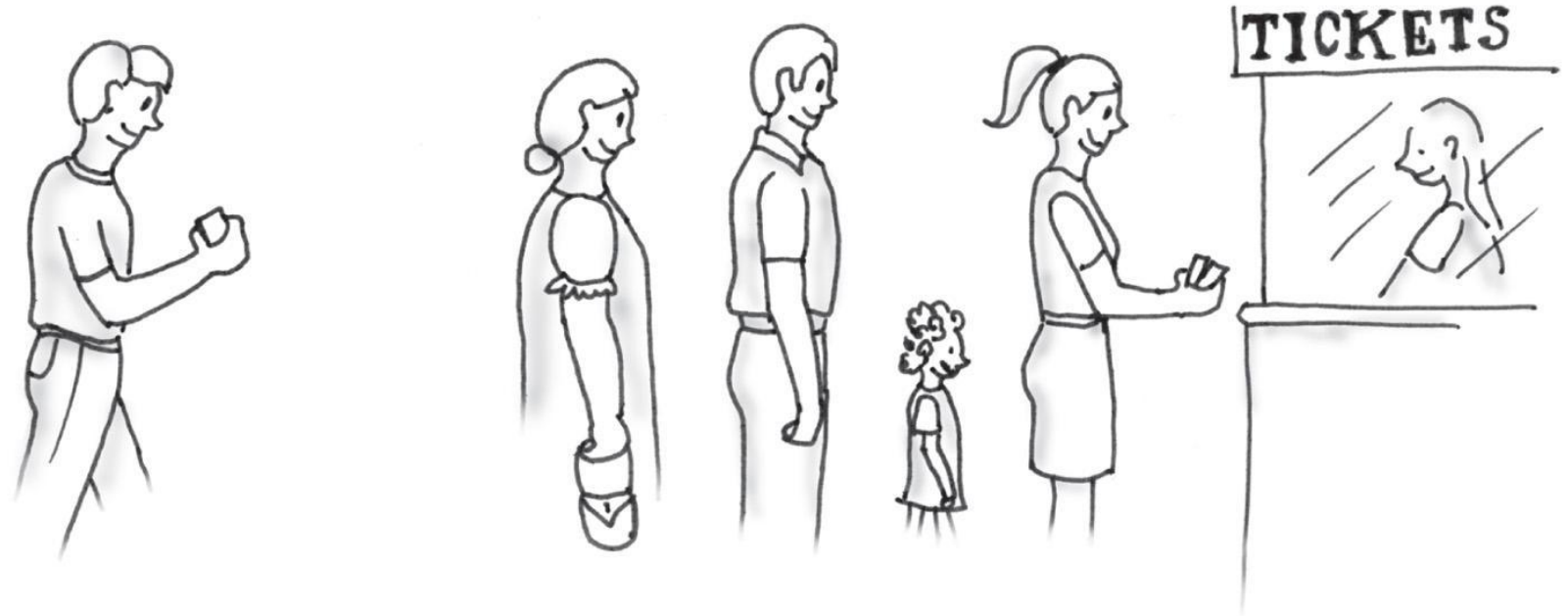
```
/** An interface for the ADT queue.  */
public interface QueueInterface<T>
{
  /** Adds a new entry to the back of this queue. */
  public void enqueue(T newEntry);

  /** Removes and returns the entry at the front of this
queue. */
  public T dequeue();

  /**  Retrieves the entry at the front of this queue. */
  public T getFront();

  /** Detects whether this queue is empty. */
  public boolean isEmpty();

  /** Removes all entries from this queue. */
  public void clear();
} // end QueueInterface
```

FIGURE 7-2 The effect of operations on a queue of strings



Queue: Simulating a Waiting Line

- A line, or queue, of people



Simulating a Waiting Line

Transaction time left: 5

Time: 0 Wait: 0

Customer **1** enters line with a 5-minute transaction.
Customer **1** begins service after waiting 0 minutes.

Transaction time left: 4

Time: 1

Customer **1** continues to be served.

Transaction time left: 3 **3**

Time: 2

Customer **1** continues to be served.
Customer **2** enters line with a 3-minute transaction.

Transaction time left: 2 **3**

Time: 3

Customer **1** continues to be served.

Transaction time left: 1 **3** **1**

Time: 4

Customer **1** continues to be served.
Customer **3** enters line with a 1-minute transaction.

Simulating a Waiting Line

Transaction time left: 3 1 2

Time: 5 Wait: 3

Customer **1** finishes and departs.
Customer **2** begins service after waiting 3 minutes.
Customer **4** enters line with a 2-minute transaction.

Transaction time left: 2 1 2

Time: 6

Customer **2** continues to be served.

Transaction time left: 1 1 2 4

Time: 7

Customer **2** continues to be served.
Customer **5** enters line with a 4-minute transaction.

Transaction time left: 1 2 4

Time: 8 Wait: 4

Customer **2** finishes and departs.
Customer **3** begins service after waiting 4 minutes.

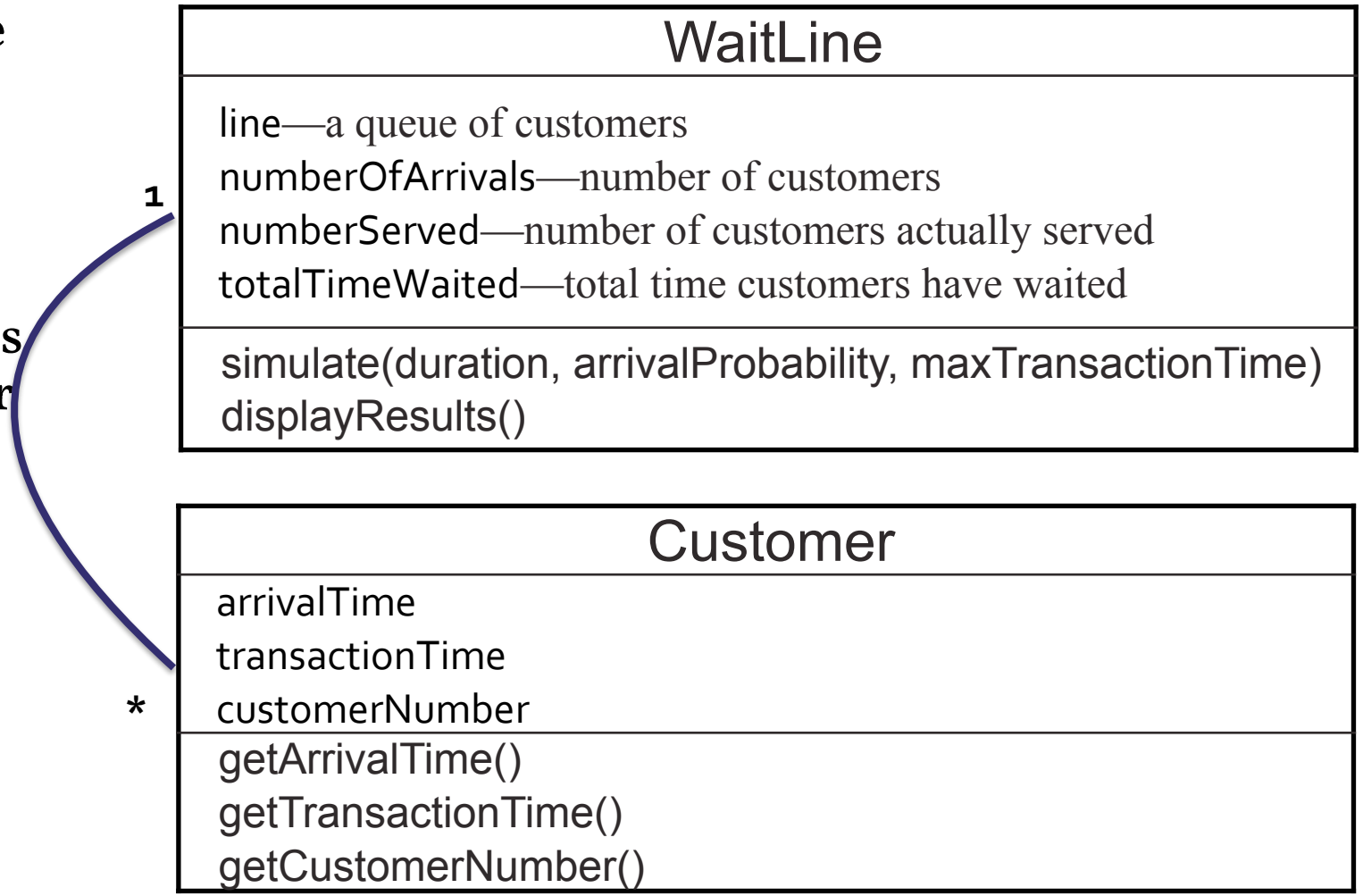
Transaction time left: 2 4

Time: 9 Wait: 4

Customer **3** finishes and departs.
Customer **4** begins service after waiting 4 minutes.

Simulating a Waiting Line

- **Problem**: Simulate one line of people waiting for service from one agent with random events.
- A diagram of the classes WaitLine and Customer



Algorithm for Simulating a Waiting Line

```
Algorithm simulate(duration, arrivalProbability, maxTransactionTime)
transactionTimeLeft = 0
for (clock = 0; clock < duration; clock++)
{
    if (a new customer arrives)
    {
        //Update numberOfArrivals
        //Create a new Customer and enqueue
    }
    if (transactionTimeLeft > 0) // If present customer is still being served
        //Update transactionTimeLeft
    else if (!line.isEmpty())
    {
        //Dequeue Customer
        //Update transactionTimeLeft, totalTimeWaited, numberServed
    }
}
```

Algorithm for Simulating a Waiting Line

```
Algorithm simulate(duration, arrivalProbability, maxTransactionTime)
transactionTimeLeft = 0
for (clock = 0; clock < duration; clock++){
    if (a new customer arrives){
        numberOfArrivals++
        transactionTime = a random time that does not exceed maxTransactionTime
        nextArrival = a new customer containing clock, transactionTime, and
                a customer number that is numberOfArrivals
        line.enqueue(nextArrival)
    }
    if (transactionTimeLeft > 0) // If present customer is still being served
        transactionTimeLeft--
    else if (!line.isEmpty()){
        nextCustomer = line.dequeue()
        transactionTimeLeft = nextCustomer.getTransactionTime() − 1
        timeWaited = clock − nextCustomer.getArrivalTime()
        totalTimeWaited = totalTimeWaited  +  timeWaited
        numberServed++
    }
}
```

Java Class Library: The Interface `Queue`

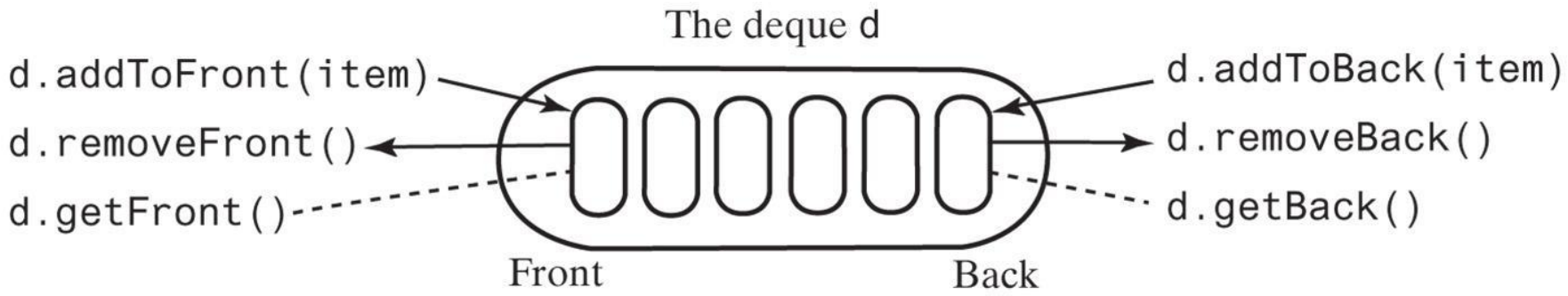
- Methods provided
  - add //Inserts the specified element into this queue
  - offer //Inserts the specified element into this queue, if enough space
  - remove //Retrieves and removes the head of this queue, exception if empty
  - poll //Retrieves and removes the head of this queue, null if empty
  - element //Retrieves, but does not remove, the head of this queue, exception if empty
  - peek //Retrieves, but does not remove, the head of this queue, null if empty
  - isEmpty
  - size

The ADT `Deque`

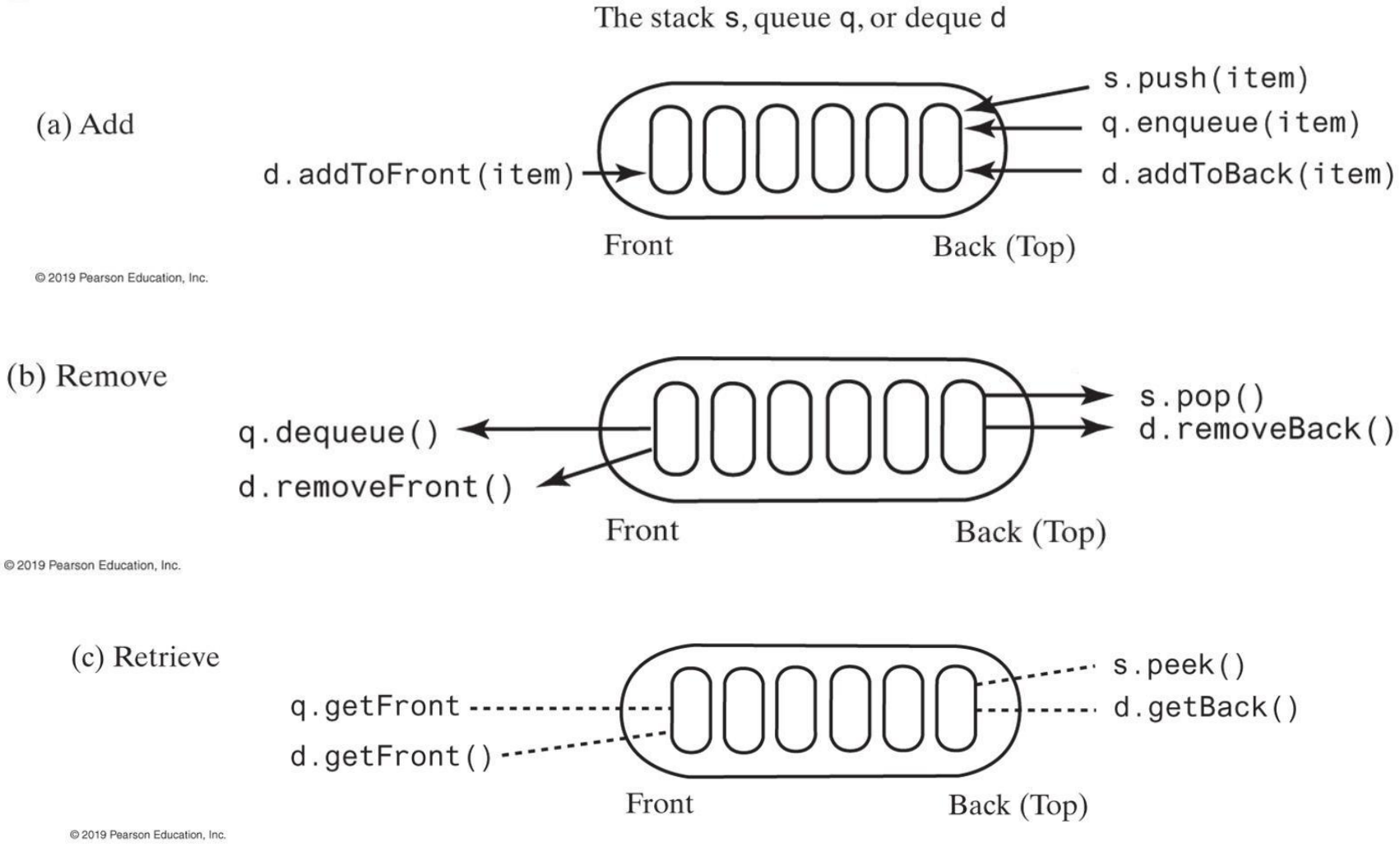
- A double ended queue
- `Deque` pronounced “deck”
- Has both queue-like operations and stack-like operations

The ADT `Deque`

- An instance d of a `deque`



Stack, Queue and Deque Comparison



Use ADT Deque to Read Keyboard Input

- When we type at keyboard, we use backspace to correct mistake. For instance: com<-mpte<-<-utr<-er should result in computer
- How would you like to use a deque to read a sequence of chars, including backspace, and output its result?

Use ADT Deque to Read Keyboard Input

```
// Read a line
d = a new empty deque
while (not end of line)
{
        character = next character read
        if (character == ←)
                d.removeBack()
        else
                d.addToBack(character)
}
// Display the corrected line
while (!d.isEmpty())
        System.out.print(d.removeFront())
System.out.println()
```

Java Class Library: The Class `ArrayDeque`

- Implements the interface **`Deque`**
- Constructors provided
  - **`ArrayDeque()`**
  - ArrayDeque(int initialCapacity)

ADT Priority Queue

- Consider how a hospital assigns a priority to each patient that overrides time at which patient arrived.
- ADT priority queue organizes objects according to their priorities
- Definition of “priority” depends on nature of the items in the queue

PriorityQueue Interface

```
/** An interface for the ADT priority queue. */
public interface PriorityQueueInterface<T extends Comparable<? super T>>
{
  /** Adds a new entry to this priority queue. */
  public void add(T newEntry);

  /** Removes and returns the entry having the highest priority. */
  public T remove();

  /** Retrieves the entry having the highest priority. */
  public T peek();

  /** Detects whether this priority queue is empty. */
  public boolean isEmpty();

  /** Gets the size of this priority queue. */
  public int getSize();

  /** Removes all entries from this priority queue. */
  public void clear();
} // end PriorityQueueInterface
```

Java Class Library: The Class `PriorityQueue`

- Basic methods
- `PriorityQueue`
  - **add**
  - remove
  - **poll**
  - element
  - **peek**
  - isEmpty
  - clear
  - getSize

Java `PriorityQueue` Discussion

Suppose lower number has higher priority:

```
PriorityQueue<Integer> myPriorityQueue = new PriorityQueue();
myPriorityQueue.add(14);
myPriorityQueue.add(10);
myPriorityQueue.add(7);
Integer number = myPriorityQueue.remove();
myPriorityQueue.add(number);
myPriorityQueue.add(16);
while (!myPriorityQueue.isEmpty())
    System.out.println(myPriorityQueue.poll());
```

1. What would be the output?
2. How would you like to implement a PriorityQueue?